PROJECT
FHP-1 Lattice Gas Simulation with MPI

Bodige Sai Vineeth Goud (2328268)
Sidhesh Prakash Jadhav (2328016)

Winter Term 2024/25

Introduction

Goal: Implement a parallelized FHP-1 lattice gas simulation using C and MPI.

- Models fluid dynamics using cellular automata.
- Simulates incompressible Navier-Stokes equations.
- Uses a hexagonal grid with six possible movement directions.

Memory Allocation

Function: allocate_memory()

```
grid = (int ***)malloc(X_NODES * sizeof(int **));
obstacle = (int **)malloc(X_NODES * sizeof(int *));
// Allocate 2D grid and obstacle arrays
```

```
for (int x = 0; x < X_NODES; x++) {
    grid[x] = (int **)malloc(Y_NODES * sizeof(int *));
    obstacle[x] = (int *)calloc(Y_NODES, sizeof(int));
    // Allocate 2D grid and obstacle arrays
}
```

Purpose:

- Allocates memory dynamically for the simulation grid.
- Sets up an obstacle array to define static barriers.

Grid Initialization

Function: initialize_grid()

```
srand(time(NULL)); // Seed the random number generator
// Initialize grid with random particle distribution
```

```
if (obstacle[x][y] == 0 && (rand() / (double)RAND_MAX) < population_density) {
    int random_direction = rand() % DIRECTIONS;
    grid[x][y][random_direction] = 1;
}
// Randomly assign particles to valid grid locations
```

Purpose:

- Initializes the simulation grid with a **random particle distribution**.
- Ensures only **valid locations** are populated.

Streaming

Function: stream()

```
for (int x = rows_start; x < rows_end; x++) {
    for (int y = 0; y < Y_NODES; y++) {
        if (obstacle[x][y]) continue;
        for (int d = 0; d < DIRECTIONS; d++) {
            int new_x = x + direction_x[d]; // Calculate new x position
            int new_y = y + direction_y[d]; // Calculate new y position
            // Ensure valid grid indices
            if (new_x >= 0 && new_x < X_NODES && new_y >= 0 && new_y < Y_NODES)
                grid[new_x][new_y][d] = grid[x][y][d];
                grid[x][y][d] = 0;
            }
        }
    }
}
```

Purpose:

- Moves particles in the grid based on their directional velocities.
- Ensures particles shift according to the FHP-1 lattice rules.

Collision Handling

Function: collision()

```
if (obstacle[x][y]) continue; // Skip obstacles
int state = 0;
// Apply FHP-1 collision rules to each grid cell
```

```
if (state == 0b000011 || state == 0b110000) {
    state = (rand() % 2) ? 0b110000 : 0b000011;
}
// Modify state based on collision rules
```

Purpose:

- Applies **FHP-1 collision rules** for each grid cell.
- Ensures **momentum conservation** while updating states.

Boundary Conditions

Function: apply_boundary_conditions()

```
for (int x = 0; x < X_NODES; x++) {
    for (int y = 0; y < Y_NODES; y++) {
        if (x == 0 || x == X_NODES - 1 || y == 0 || y == Y_NODES - 1) {
            // Apply periodic boundary conditions (wrap around grid edges)
            for (int d = 0; d < DIRECTIONS; d++) {
                grid[x][y][d] = grid[(x + X_NODES - 1) % X_NODES][(y + Y_NODES
            }
        }
    }
}
```

Purpose:

- **Periodic boundary conditions** to wrap particles around at grid edges.
- Ensures particles exiting one edge re-enter from the opposite edge.

Density Calculation

Function: calculate_density()

```
double total_particles = 0;
for (int x = 0; x < X_NODES; x++) {
    for (int y = 0; y < Y_NODES; y++) {
        if (obstacle[x][y]) continue;
        for (int d = 0; d < DIRECTIONS; d++) {
            total_particles += grid[x][y][d];
        }
    }
}
double density = total_particles / (X_NODES * Y_NODES);
```

Purpose:

- **Calculates particle density** in the entire simulation grid.
- Density is computed as the ratio of total particles to grid size.

Velocity Calculation

Function: calculate_velocity()

```
double total_velocity_x = 0, total_velocity_y = 0;
for (int x = 0; x < X_NODES; x++) {
    for (int y = 0; y < Y_NODES; y++) {
        if (obstacle[x][y]) continue;
        for (int d = 0; d < DIRECTIONS; d++) {
            total_velocity_x += grid[x][y][d] * velocity_x[d];
            total_velocity_y += grid[x][y][d] * velocity_y[d];
        }
    }
}
double velocity = sqrt(total_velocity_x * total_velocity_x + total_velocity_y *
```

Purpose:

- **Calculates the velocity** of particles in the grid.
- The total velocity is computed based on the directional movement of particles.

Results and Visualization

- Outputs saved for visualization in MATLAB.
- Macroscopic flow analyzed using density and velocity plots.
- Simulation correctness verified against theoretical expectations.

Conclusion

- Successfully implemented an MPI-based FHP-1 lattice gas model.
- Parallelization improves computational efficiency.
- Future work: Performance optimizations and larger simulations.